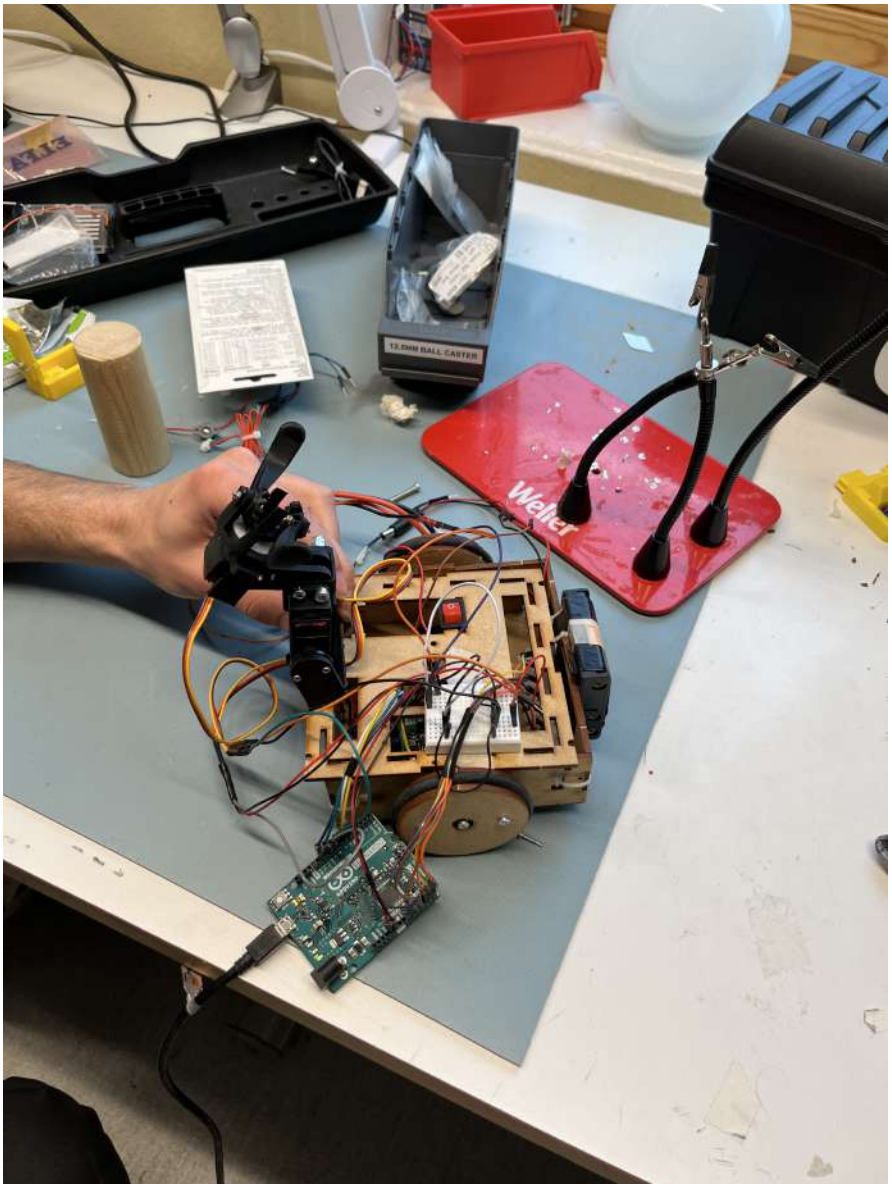
Service Robotics - Project Report

TSN01

Group 6

Alexander Anvin | **Kristoffer Eriksson** | **Alex Hirsch** | **Markus Lantto**

December 2025



1 Preface

All members worked on all of the different parts of the robot. Markus did most of the CAD and printing for the parts added to the robot as well as wiring for the base cuboid. Alex did the CAD and printing of the claw and its wiring. Alexander, Kristoffer and Alex worked on the maze solving algorithm which was the part that took the longest while Markus did the designing and printing of the backpanels and storage for the cylinders. All members were apart of a lot of hours testing the robot in the maze too se it could handle different cases and running speeds.

Contents

1 Preface **2**

2 Introduction **3**

3 Robot system description **3**
3.1 Physical design 3
3.2 Electronics design 4
3.3 Algorithm design 5

4 Result **5**

5 Discussion **6**

6 Appendix **7**

2 Introduction

This is the project report for Group 6 in the course Service Robotics at LTH. The aim of the project was to design, build, and validate a mobile robotics system that uses sensors to successfully navigate, locate, and retrieve three wooden cylinders. The task was carried out by integrating an Arduino Leonardo microcontroller together with sensors, servos, motors, and different building parts, such as mechanical components manufactured with laser cutters and 3D printers. By experimenting with different combinations of mechanical solutions, electronics, and code, a fully functioning rescue robot was created. The final prototype was then evaluated in terms of navigation, robustness, speed, and its ability to consistently locate and retrieve the cylinders. The results show that the robot system successfully meets the project course requirements.

3 Robot system description

3.1 Physical design

The physical design of the robot is based on the material obtained at the start of the course. We have a standard cuboid as the base where we keep the wheel motors inside as well as the majority of the cables to keep the design clean. On the sides of the cuboid we have mounted two wheels on opposite sides. We have the IR-sensors mounted underneath the front of the robot to detect the line. The claw is mounted on the upper end of the base cuboid facing forward so we can grab the cylinders. On top of the base cuboid there is a sloped container used for storing the cylinders we have picked up. We are using two different battery packs, one for the motors and one to power the arduino and other mechanisms. To keep the design clean and to be able to use as much as the wire inside we have stored both battery packs on the back of the robot having panels to attach them.

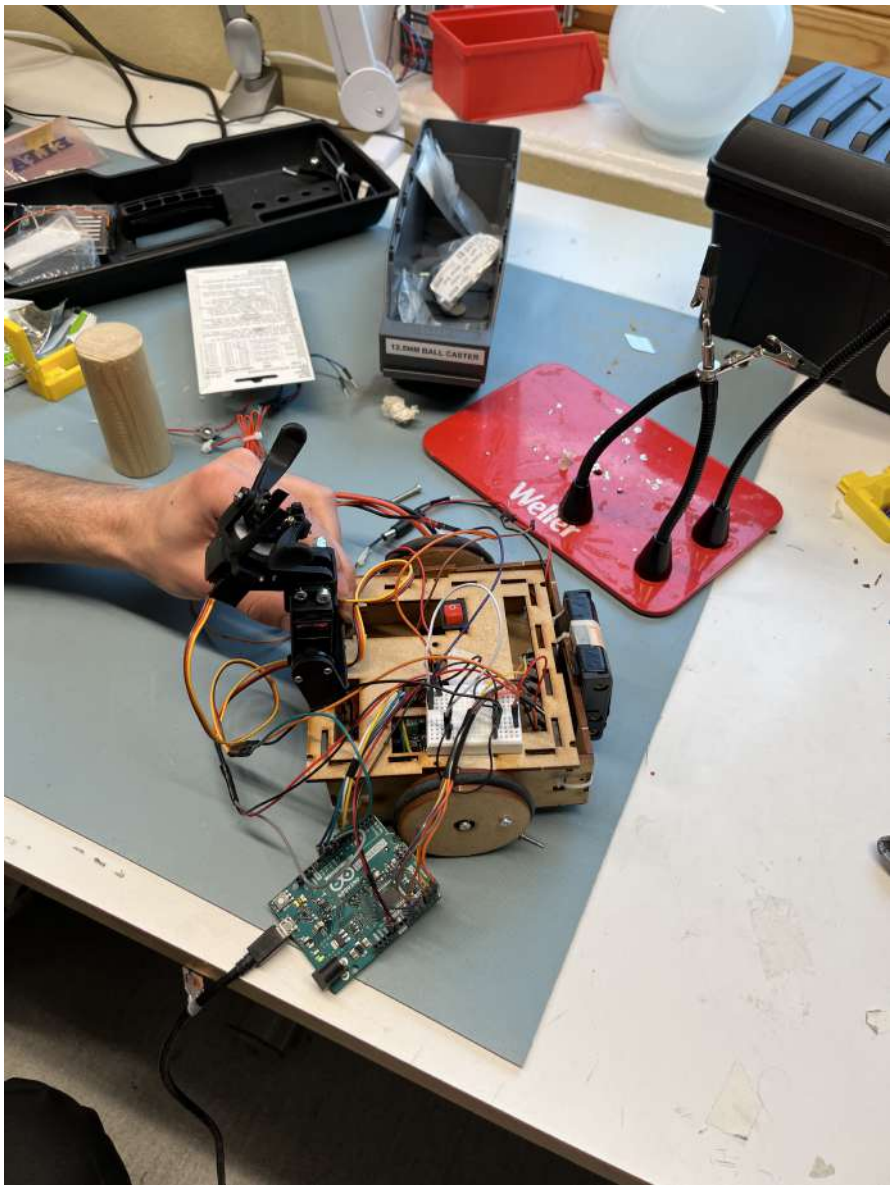


Figure 1: The robot design before adding the top part

3.2 Electronics design

The electronics design we went for was based on the electrical components that were provided. Two DC motors, one per wheel drive, are connected to the motor driver. The motors are regulated by using digital PWM signal pins, in order to control the speed and direction of the wheels. The claw mechanism is operated by two servo motors, where one controls the vertical lift of the claw and the other one controls the opening and closing of the gripper fingers. Servos are controlled by the Arduino with digital PWM pins, to set the desired angular position. An IR reflectance sensor is mounted underneath the front of the robot, which allows for line detection and following. This sensor ensured that the robot always stayed in the center of the maze corridors, and detect curves and junctions. The claw has a pressure sensor mounted in the center of the claws in order to detect and grab victims.

The robot was powered by a dual battery system, in order to provide sufficient and isolated power to the different systems. A separate high-capacity battery pack was dedicated to powering the DC motors for the wheels, through a motor driver. The power source was a powerbank that powered the arduino itself, along with the sensors and servo motors.

3.3 Algorithm design

The algorithm we have implemented to be able to go through the whole maze is based on a map over the maze and a BFS search. Since we know how the maze looks we have elected to build a map over the maze as a graph data structure. Where all turns and intersections are handled as nodes and the path between two nodes as edges. We have an adjacency matrix which is a 2d vector where the first dimension is the node, and the second is what direction the robot is currently facing. This means we are always keeping track of our current node and current direction. The algorithm then utilizes this map to implement a BFS-algorithm to go from a starting node to a target node. We elected to use a BFS since we have an unweighted graph and therefore BFS guarantees to find the shortest path. The algorithm does not itself manage the wheels or instructions to the robot but it creates a stack with commands from which the robot pops when it gets to a node. E.g if the robot is supposed to take two right turns, go straight in an intersection and then turn left, our algorithm will generate a stack looking like, R,R,S,L. The main loop then works so that the robot goes straight until the IR-sensors underneath senses that there is black tape on either the left or the right. It then pops the first instruction on the top of the stack and executes that by looking if it's R or L and then turning 90 degrees and then also updating the current heading. If it's an S then it continues onward until the next occurrence of black tape on left or right. Using this method, solving the spots where the tape is missing is not that difficult. Since the robot always keeps track of its current node and what node that is adjacent to its current node, the logic for how to handle the case when there is no tape changes when we are about to enter the gap. When this logic is used the priority becomes re-aquiring the line to center the robot and then it either goes straight until it finds it or does a normal turn and goes straight until it finds the line.

4 Result

The final rescue robot prototype was tested during the course's final robot demo by evaluating its performance in navigating and rescuing the wooden cylinders. After an initial failed run, our robot successfully mapped the maze and completed the stated mission by retrieving all 3 "victims", although one intervention was necessary since the final victim was dropped on the pickup. The final time for our demo landed at 1 minute 52 seconds, which gave us 208 points.

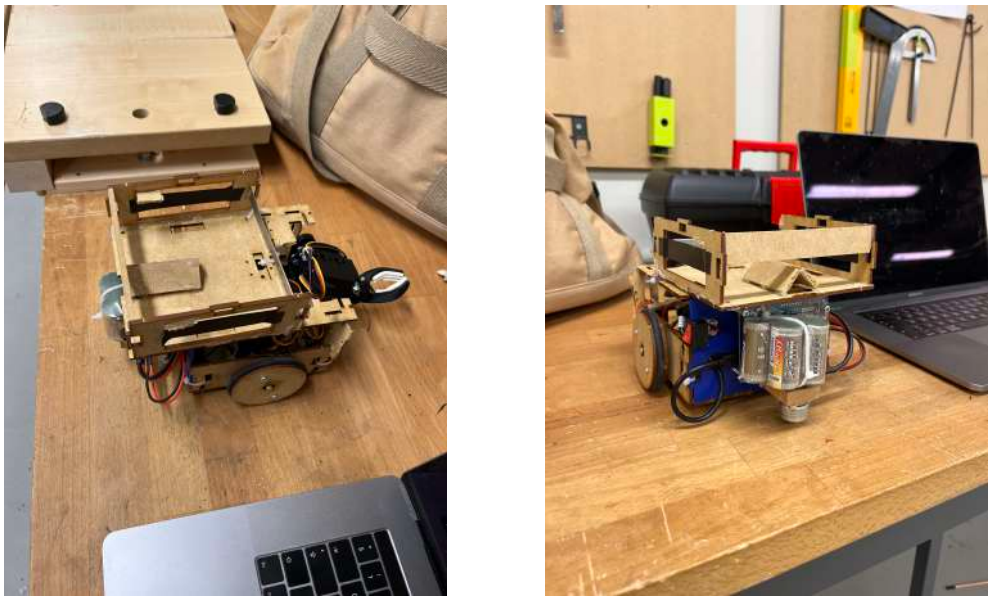


Figure 2: Pictures of the final robot design

5 Discussion

We went for the cuboid base design since the building parts were already laser cutted and provided for us, but also because it initially seemed to allow for maximum space of internal motors while still providing a low center of gravity. It kept the motors from potential external damage and had enough space over for the rest of the electrical components. In the beginning our idea was to implement a return logic for the robot so when the third cylinder had been grabbed the navigation logic would switch and return home with the fastest path possible. We had this code implemented as our SolveMaze method from the current node to the start node would guarantee that we would return as fast as possible. However during testing we got the time to go through the whole maze to a constant of 1 minute and 59 seconds. Seeing as we could guarantee this time every time we did the complete maze and back to the start node but going from a current node to 0 could lead to untested scenarios depending on what node the current on was, we elected to simply go through the whole maze no matter when we picked up the cylinders. We came to the conclusion that the added risk of being in an untested scenario was not worth the reward of the extra seconds faster as we already could guarantee to rescue all three cylinders in the time frame. In the latter stages of the project, we decided to improve on the design of the building parts of our rescue robot, with the specific goals to implement storage for the cylinders, change the center of mass to be further back by adding a battery backpack, and to add some screw holes for mounting the microcontroller to improve the wire flow. For this we 3D printed three new parts, a back panel, top panel, and cylinder container. Since we ran out of time and couldn't pick up our prints due to X-labs' narrow opening hours, only the back panel was integrated into our build. Instead we used more MDF to achieve the same goals stated above, which worked satisfactory, but didn't come out as robust as the prints may have been. Which led to a small malfunction on the gripper function in the first run due to the MDF-geometry that the gripper housing was mounted on had lost some of its glue and as a consequence slipped a tiny angle when a cylinder was grabbed. This led to a failed first run, but after some quick tweaks our second run was successful. If we were to redo the project, we probably would've focused more on the CAD designs earlier on in the course since 3D-prints often need a few iterations to actually provide any improvements on a build.

6 Appendix

This was our main loop in the code

```
void loop() {
    if (returningHome) {
      processReturnJourney();
      return;
    }

    uint16_t position = qtr.readLineBlack(sensorValues);
    bool farLeft  = sensorValues[0] > Threshold;
    bool farRight = sensorValues[5] > Threshold;

    bool allWhite = true;
    for(int i=0; i<6; i++) { if(sensorValues[i] > 400) allWhite =
        false; }

    // PRIORITY 1: RE-ACQUIRING LINE AFTER GHOST NODE
    if (!allWhite && wasInGap) {
      wasInGap = false;

      int targetNode = pathNodeIDs[pathIndex];
      Serial.print("GAP␣ENDED␣at␣Node:␣"); Serial.println(
          targetNode);

      // SPECIAL CASE: Node 13 (Keep your existing logic for N13
          if needed)
      if (targetNode == 13) {
         pathIndex++;
      }

      // --- FIX: ALWAYS STABILIZE ---
      // We apply this to ALL nodes after a gap (including Node
          6), not just specific ones.
      // This forces the robot to straighten out before it looks
          for turns.

      unsigned long stabTime = millis();
      while(millis() - stabTime < 350) { // 350ms of pure line
          following
         uint16_t pos = qtr.readLineBlack(sensorValues);
         int err = pos - 2500;
         int adj = (Kp * err) + (Kd * (err - lastError));
         lastError = err;

         // Use slightly gentler speeds for stabilization if
             necessary
         int l = M2_BaseSpeed + adj;
         int r = M1_BaseSpeed - adj;
         l = constrain(l, -100, 255); r = constrain(r, -100, 255)
             ;
         runMotors(l, r);
      }

      // CRITICAL: Return here so we don't accidentally trigger '
          handleIntersection'
      // in the same loop cycle.
      return;
  }

    // PRIORITY 2: INTERSECTIONS
    if (farLeft || farRight) {
        wasInGap = false;
```

```
        handleIntersection();
        return;
    }

    // PRIORITY 3: GAP / DEAD END HANDLING
    if (allWhite) {

      // --- TARGET REACHED CHECK ---
      if (pathIndex >= pathSequence.length()) {
          Serial.println("TARGET REACHED. Initiating Return.");
          runMotors(0,0);
          delay(500);

          for(int k=0; k<5; k++) {
            digitalWrite(LED_BUILTIN, HIGH); delay(100);
            digitalWrite(LED_BUILTIN, LOW); delay(100);
          }

          turnAround180();
          returningHome = true;
          lastError = 0;
          return;
      }

      // [FIX] DEAD END HANDLING
      // If the sensors see white (end of line) AND the current instruction is 'U'
      // We treat this as arriving at the node and executing the U-turn.
      char nextMove = pathSequence.charAt(pathIndex);
      if (nextMove == 'U') {
          Serial.println("Dead End Detected. Executing Planned U-Turn.");
          turnAround(); // Perform the 180
          pathStack[stackPointer++] = 'U'; // Add to history
          pathIndex++; // <--- CRITICAL: Increment index so we don't repeat this move at the next node!
          wasInGap = false;
          lastError = 0;
          return;
      }

      int prevNode = pathNodeIDs[pathIndex - 1];
      int nextNode = pathNodeIDs[pathIndex];
      int nextNextNode = -1;
      if (pathIndex + 1 < MAX_NODES) nextNextNode = pathNodeIDs[pathIndex + 1];
```

Aswell as our BFS-algorithm

```
String solveMaze(int start, int end, int &heading) {
int queue[MAX_NODES], parent[MAX_NODES];
bool visited[MAX_NODES];
int head = 0, tail = 0;

for(int i=0; i<MAX_NODES; i++) { visited[i] = false; parent[i] = -1; }

queue[tail++] = start; visited[start] = true;

bool found = false;
while(head < tail) {
  int curr = queue[head++];
  if (curr == end) { found = true; break; }
```

```
14
15        for (int dir=0; dir<4; dir++) {
16          int neighbor = mazeMap[curr][dir];
17          if (neighbor != -1 && !visited[neighbor]) {
18            visited[neighbor] = true; parent[neighbor] = curr;
                  queue[tail++] = neighbor;
19          }
20        }
21      }
22
23      if (!found) return "";
24
25      int path[50]; int count = 0; int curr = end;
26      while(curr != -1) { path[count++] = curr; curr = parent[curr
            ]; }
27
28      // --- CHANGED: APPEND NODES TO GLOBAL ARRAY WITHOUT
            OVERWRITING ---
29      // We skip the first node (k=1 instead of k=0) for chained
            segments
30      // so we don't record the connecting node twice (e.g. 0..29,
            29..26)
31      // UNLESS it is the very first segment (nodeFillIndex == 0).
32
33      int startK = (nodeFillIndex == 0) ? 0 : 1;
34
35      for(int k = startK; k < count; k++) {
36          // Logic: path array is backwards (End->Start), so we
                read it in reverse
37          pathNodeIDs[nodeFillIndex++] = path[count - 1 - k];
38      }
39
40      String strPath = "";
41      int currentHeading = heading; // Use the heading passed from
            setup
42
43      // Generate Path String
44      for (int i = count - 2; i >= 0; i--) {
45          int from = path[i+1]; int to = path[i];
46          int travelDir = -1;
47
48          if (mazeMap[from][NORTH] == to) travelDir = NORTH;
49          else if (mazeMap[from][EAST] == to) travelDir = EAST;
50          else if (mazeMap[from][SOUTH] == to) travelDir = SOUTH;
51          else if (mazeMap[from][WEST] == to) travelDir = WEST;
52
53          int diff = travelDir - currentHeading;
54          if (diff < 0) diff += 4;
55
56          if (diff == 0) strPath += "S";
57          else if (diff == 1) strPath += "R";
58          else if (diff == 3) strPath += "L";
59          else if (diff == 2) strPath += "U";
60
61          currentHeading = travelDir; // Update local tracking
62      }
63
64      // --- VITAL: Update the global heading for the next function
             call ---
65      heading = currentHeading;
66
67      return strPath;
68    }
```